PRD: Design-to-Exit — Solo Founder Business Transformation App

Status: Ready for implementation
Target: PoC / Demo (single user)

Problem Statement

Solo founders — coaches, consultants, creatives, doctors — are generating revenue but are completely dependent on themselves to deliver it. They have demand but no system. No predictability. No scalability. No path to exit. They don't know what moves to make, in what order, to transform their personal income source into a sellable business asset.

Solution

A guided web app that walks founders through a 16-step framework, one step at a time, to redesign their business into an exitable company. Each step asks targeted questions about their current situation, then uses an LLM to suggest a tailored "Exitable Design Move" and personalized action plan. After completing all 16 steps, the app generates a shareable pitch deck that presents their transformed business to potential investors or acquirers.

User Stories

1. As a solo founder, I want to see all 16 framework steps at a glance in a sidebar, so that I understand the full journey I'm about to take.
2. As a solo founder, I want steps to unlock sequentially, so that I build context progressively before tackling each move.
3. As a solo founder, I want each step to explain what it's about before asking me questions, so that I understand why it matters.
4. As a solo founder, I want to answer multiple focused sub-questions per step with helpful placeholders, so that I understand what quality answers look like.
5. As a solo founder, I want the placeholder text to show me the optimal answer format, so that I'm guided without feeling constrained.
6. As a solo founder, I want to submit my answers and receive an LLM-suggested "Exitable Design Move", so that I learn how to reframe my current situation.
7. As a solo founder, I want the suggested design move to account for everything I've answered in previous steps, so that the advice is coherent and cumulative.

8. As a solo founder, I want to edit the suggested design move, so that I can refine it to match my actual business.
9. As a solo founder, I want to receive a personalized “How to do it” action plan per step, so that I know the concrete next steps to execute the design move.
10. As a solo founder, I want the “How to do it” to be specific to my business context, not generic advice.
11. As a solo founder, I want to see my accepted design move saved on the step card, so that I can review my progress.
12. As a solo founder, I want to return to any completed step and edit my answers or design move, so that my thinking can evolve over time.
13. As a solo founder, I want my progress to be automatically saved, so that I don’t lose my work between sessions.
14. As a solo founder, I want a clear visual indicator of which steps are completed, in progress, or locked, so that I always know where I am.
15. As a solo founder, I want to see a summary of all my design moves as I complete each step, so that I can see the business I’m building.
16. As a solo founder, I want a “Generate Pitch Deck” button that appears after I complete all 16 steps, so that I know when I’m ready to produce my output.
17. As a solo founder, I want the pitch deck generation to incorporate all my design moves and business context, so that it tells a coherent investment story.
18. As a solo founder, I want the pitch deck to be structured as a real investor presentation with clear slides, so that I can share it with potential investors or acquirers.
19. As a solo founder, I want the pitch deck available at a dedicated URL, so that I can share it easily.
20. As a solo founder, I want the pitch deck to include: Cover, Problem Statement, Market Opportunity, TAM/SAM/SOM, Solution, Operating Model, Value Creation Process, Business Model & Traction, Milestones, Go-to-Market, Competitive Advantage, Team, Impact, Ask, and Thank You slides.
21. As a solo founder, I want each pitch deck slide to use the content I built across the 16 steps, so that the deck is grounded in real analysis, not generic filler.
22. As a solo founder, I want the pitch deck to look professional enough to share with investors, so that it creates a credible first impression.

Implementation Decisions

Modules

1. Framework Data Module (static)
Central source of truth for all 16 steps. Each step entry contains: step number, title, description, sub-questions (derived from framework columns: current state, market context, financial situation, assets/capabilities), optimal answer placeholders, and the pitch slide this step feeds into. No runtime dependency — pure static data.

2. State Manager
Thin wrapper around `localStorage`. Stores per-step state: user’s raw answers to sub-questions, accepted design move text, how-to-do-it text, and completion status. Exposes typed read/write/reset operations. All 16 steps’ state accumulates here and serves as the LLM context payload.

3. LLM Client
Anthropic Claude Sonnet 4.6 API wrapper. Two call types: - **Step call**: receives current step definition + user's answers to this step + all previous steps' answers and accepted design moves → returns `{ designMove: string, howToDoIt: string[] }`
- **Pitch call**: receives all 16 completed steps (answers + design moves) → returns structured `PitchDeckJSON` (one object per slide with typed content fields matching the template)

System prompt establishes the framework's philosophy: solo operator → exitable company. Context grows with each step.

4. Step Screen
Single page/component that renders any of the 16 steps. Sections: step header (number, title, description), sub-questions with inputs and placeholders, submit button (triggers LLM step call), LLM output section (editable design move textarea + how-to-do-it list). Completed state shows saved design move with edit affordance.

5. Sidebar
Persistent navigation showing all 16 steps. Three states per step: locked (grey, not clickable), unlocked/current (active), completed (checkmark, clickable to revisit). Shows step number and title. Highlights current step.

6. Pitch Deck Renderer
`/pitch-deck` route. Reads pitch JSON from localStorage (written after generation). Renders 15 slides as full-screen CSS snap-scroll sections. Inspired by the PDF template structure — same slide titles and content hierarchy, clean Tailwind implementation. No LLM call at render time; purely presentational.

Sub-question Derivation

Each step's sub-questions map to the relevant framework columns for that step. Not every column applies to every step. Core questions extracted from: - **Current version column** → "How would you describe your current [step topic]?"
- **Financial Model column** → revenue/pricing questions where relevant (Steps 1–7)
- **Market column** → market sizing questions where relevant (Steps 1–3, 10)
- **Assets column** → what exists today that could be formalized (Steps 8–11)
- **Exit/Investor columns** → forward-looking questions (Steps 12–16)

LLM Context Strategy

Each step call passes: 1. System prompt: framework philosophy + role (business transformation advisor) 2. Business context: all previous steps' sub-question answers + accepted design moves in structured format 3. Current step: definition, example design move from framework (as inspiration, not template), user's answers

The LLM is instructed to output the design move as a single punchy sentence (like the framework examples) and how-to-do-it as 3–5 concrete action items.

Pitch Deck JSON Schema

```
type PitchDeckJSON = {
  cover: { companyName: string; valueProposition: string;
        targetAudience: string; founderName: string }
  problem: { headline: string; clarifyingParagraph: string;
        facts: Array<{ headline: string; data: string }> }
  opportunityGap: { marketGap: string; supportingParagraph:
        string }
  opportunitySize: { tam: string; sam: string; som: string }
  solution: { description: string; points: string[] }
  operatingModel: { columns: Array<{ headline: string; paragraph:
        string }> }
  valueCreation: { headline: string; paragraph: string }
  businessModel: { revenueStreams: string; tractionEvidence:
        string }
  milestones: Array<{ year: string; objective: string }>
  goToMarket: { columns: Array<{ headline: string; points:
        string[] }> }
  competitiveAdvantage: { headline: string; description: string }
  team: { members: Array<{ name: string; role: string;
        description: string }> }
  impact: { points: string[]; vision: string }
  ask: { amount: string; useOfFunds: string; paragraph: string }
  thankYou: { message: string; contactName: string }
}
```

State Persistence

All state in `localStorage` under a single key (`gia-design-to-exit-state`). Shape:

```
type AppState = {
  steps: Record<number, {
    answers: Record<string, string>   // questionId → user answer
    designMove: string                // accepted (possibly
        edited) design move
    howToDoIt: string[]              // LLM-generated action items
    completedAt: string              // ISO timestamp
  }>
  pitchDeck: PitchDeckJSON | null
}
```

Tech Stack

- **Framework**: Next.js (App Router)
- **Styling**: Tailwind CSS
- **LLM**: Anthropic SDK (`@anthropic-ai/sdk`), Claude Sonnet 4.6
- **State**: localStorage (no database, no auth)
- **API key**: `ANTHROPIC_API_KEY` environment variable, called from Next.js API route (not exposed to client)

Testing Decisions

What makes a good test here: Test behavior visible to the user or to calling code — not implementation internals. A good test says "given this input, this output appears" without caring how.

Modules to test:

1. **Framework Data Module** — verify all 16 steps are present, each has required fields (title, description, sub-questions, pitch slide mapping), no missing placeholders. Pure data validation, no mocks needed.

2. **State Manager** — unit tests: write step state, read it back, update design move, mark complete, reset. Mock localStorage with `jest.spyOn` or `localStorage` polyfill.

3. **LLM Client** — integration-style unit tests with mocked Anthropic SDK. Verify correct prompt structure is sent (system prompt present, prior context included, step definition present). Verify output is parsed correctly into `{ designMove, howToDoIt }` shape.

4. **Pitch Deck JSON schema** — validate LLM pitch output against the `PitchDeckJSON` type at runtime (zod schema) so bad LLM output fails fast with a clear error rather than silently breaking the UI.

No UI component tests for the PoC — manual testing sufficient given single-user scope.

Out of Scope

- User authentication and multi-user support
- Database persistence (future migration path: replace localStorage state manager with API calls)
- Shareable pitch URL with unique ID (single `/pitch-deck` route sufficient for PoC)
- Pixel-perfect PDF template recreation
- Mobile-optimized layout
- Pitch deck export to PDF or PowerPoint
- Email/notification features
- Payment or subscription gating
- Admin panel or analytics

Further Notes

- The framework is intentionally built around a photography business as the running example. The LLM prompt must make clear this is just illustrative — the app serves all solo founder types (coaches, consultants, doctors, creatives).
- Step 1 captures initial business context (name, industry, revenue) — no separate onboarding screen.

- The “How to do it” section is purely LLM-generated per step, personalized to the user’s answers. No static framework text shown.
- Future: when multi-user support is added, the localStorage state manager interface should remain identical — only the implementation swaps to API calls. Design the state manager with this migration in mind.
- Anthropic API key must be stored server-side only, called from a Next.js API route. Never expose to client bundle.